Nombre: Morales Gonzales Emmanuel – Boleta: 2025630602, Romero Sayún Jonathan - Boleta: 2025630298 ° Grupo: 3CV4

Ejercicio 1, Parte A — Control de versiones con Git y GitHub

1. Qué es un sistema de control de versiones y qué problema resuelve

Un sistema de control de versiones (SCV) es una herramienta que registra, de forma incremental y reversible, los cambios hechos sobre un conjunto de archivos a lo largo del tiempo, de modo que sea posible recuperar cualquier estado anterior, saber quién introdujo cada modificación y por qué (Chacon & Straub, 2014). Git pertenece a la familia de los sistemas distribuidos: cada colaborador posee una copia íntegra del historial, no sólo del estado más reciente, por lo que puede trabajar y consultar el pasado del proyecto sin conexión al servidor central.

El problema concreto que resuelve en un trabajo en equipo es la **integración de cambios concurrentes sin pérdida de trabajo ni bloqueo mutuo**. Sin un SCV, dos personas que editan el mismo archivo terminan intercambiando copias (informe_final_v2_JUAN_BUENO.docx), y la última que guarda sobreescribe a la otra; nadie sabe qué versión es la vigente ni qué cambió entre una y otra. Con Git, cada colaborador trabaja sobre su propia línea de desarrollo, el sistema combina automáticamente las modificaciones que no se solapan y señala explícitamente las que sí, exigiendo una decisión humana. Además, el historial queda como registro auditable: si una funcionalidad dejó de operar, se puede identificar la confirmación exacta que la rompió y revertirla.

2. Diferencia entre Git y GitHub

Git es el software de control de versiones: un programa que se ejecuta localmente y que implementa el modelo de confirmaciones, ramas y fusiones. Es libre, funciona sin red y no pertenece a ninguna empresa.

GitHub es un servicio comercial de alojamiento de repositorios Git en la nube, propiedad de Microsoft, que añade sobre Git una capa de colaboración que Git no tiene: interfaz web, control de acceso, issues, Pull Requests, revisión de código, integración continua y documentación.

La analogía útil: Git es al control de versiones lo que el sistema de archivos es al almacenamiento; GitHub es un servicio que aloja repositorios, como Google Drive aloja archivos. Se puede usar Git sin GitHub (con un servidor propio, o sin servidor alguno), y existen alternativas equivalentes a GitHub: GitLab, Bitbucket, Gitea o Codeberg. Por lo

tanto, no son sinónimos: uno es la tecnología, el otro es una plataforma construida sobre ella.

3. Definiciones con ejemplo

- **Repositorio**. Directorio de trabajo más la carpeta oculta. git, donde se almacenan el historial completo, las ramas y la configuración. Ejemplo: la carpeta practica1-bd/ de esta práctica, con todos sus documentos y su historial de confirmaciones.
- **Confirmación (commit).** Instantánea del estado del proyecto en un momento dado, identificada por un resumen criptográfico (SHA-1/SHA-256), con autor, fecha, mensaje y un enlace a su confirmación padre. Ejemplo: a3f9c21 — "Agrega compose.yaml con volumen nombrado para PostgreSQL 16".
- **Rama (branch).** Apuntador móvil a una confirmación, que permite desarrollar una línea de trabajo independiente sin afectar a las demás. Ejemplo: crear feature/diagrama-er para construir el modelo entidad-relación mientras main conserva la versión entregable.
- **Fusión (merge).** Operación que integra el trabajo de una rama en otra, generando una confirmación con dos padres. Ejemplo: fusionar feature/diagrama-er en main cuando el diagrama queda terminado.
- **Conflicto de fusión.** Situación en la que dos ramas modificaron las mismas líneas del mismo archivo y Git no puede decidir cuál prevalece; marca el archivo y detiene la fusión hasta que una persona resuelva. Ejemplo: dos integrantes editan el índice del README.md; Git inserta marcadores <<<<<<>>>>>> y espera la resolución manual.
- **Pull Request (PR).** Solicitud formal, en la plataforma de alojamiento, para incorporar los cambios de una rama en otra. No es parte de Git: es una función de GitHub. Abre un espacio de discusión, revisión línea por línea y verificación automática antes de fusionar. Ejemplo: «PR #3: Agrega el modelo entidad-relación del consultorio» con la descripción de las entidades introducidas.
- **Archivo .gitignore**. Lista de patrones de archivos que Git debe ignorar deliberadamente, porque son generados, voluminosos o contienen secretos. Ejemplo: en esta práctica se ignoran .env, pgdata/ y .DS_Store; versionar el directorio de datos de PostgreSQL sería un error grave, porque son archivos binarios que cambian en cada escritura.
- **Archivo README.md**. Documento en formato Markdown que la plataforma muestra al abrir el repositorio; describe qué es el proyecto, cómo se usa y

cómo se navega. Ejemplo: el README.md de este repositorio, con los datos del alumno y el índice de la práctica.

4. Flujo de trabajo basado en ramas y revisión entre pares

Un flujo basado en ramas (branching workflow) establece que la rama principal (main) debe mantenerse siempre en un estado estable y entregable, y que todo trabajo nuevo se realiza en una rama temporal creada a partir de ella. El ciclo típico es: crear la rama con un nombre que describa su propósito (feature/, fix/, docs/), confirmar en ella los cambios, publicarla en el servidor, abrir un Pull Request, atender los comentarios de la revisión, fusionar y eliminar la rama. Variantes formalizadas de este esquema son GitHub Flow, Git Flow y trunk-based development, que se distinguen por cuántas ramas de larga vida admiten.

La revisión entre pares antes de fusionar responde a tres razones. La primera es de **calidad**: un segundo lector detecta errores lógicos, casos no contemplados y decisiones de diseño cuestionables que el autor, habituado a su propio código, ya no ve. La segunda es de **difusión del conocimiento**: la revisión evita que una parte del sistema sea comprendida por una sola persona, lo que reduce el riesgo de dependencia individual. La tercera es de **trazabilidad**: el Pull Request deja registrado por qué se hizo un cambio, discusión que resulta más valiosa meses después que el propio diff. En un contexto de bases de datos esto es especialmente crítico, porque un cambio de esquema mal revisado puede provocar una pérdida de datos irreversible en producción.

Referencias

Chacon, S., & Straub, B. (2014). Pro Git (2.ª ed.). Apress. https://git-scm.com/book

GitHub, Inc. (2024). GitHub Docs: About pull requests. https://docs.github.com/pull-requests

Spinellis, D. (2012). Git. IEEE Software, 29(3), 100–101. https://doi.org/10.1109/MS.2012.6